# Fundamentals of Machine Learning

### Understanding, Proofs and Experiments: Problems with Solutions

Part I: understanding questions and proofs. Part II: real-world situations and experiments to read, diagnose and design.

## How to use this set

**Part I** is about understanding and mathematics. It opens with eight short questions that each want a precise two-or-three-sentence answer, the kind an examiner uses to check whether a definition has been understood rather than memorised. Then four problems derive results the course uses without proving: why cross-entropy training produces probabilities, how much data model selection really needs, why LDA and Naive Bayes are secretly linear classifiers, and why no classifier is best at every prevalence.

**Part II** is about situations. Each problem gives you an experiment or a deployment the way it would arrive at work: a table of results, a protocol someone followed, a number someone reported. Your job is to read it, say what it does and does not show, compute what can be computed, and design the next experiment.

Each problem opens with a blue box holding the setup, then a yellow box listing the tools with a pointer [one-read §x] to the one-read document. Each sub-question is followed by its full solution. Difficulty: ⋆ direct, ⋆⋆ several steps, ⋆⋆⋆ derivation or design. All logarithms are natural.

## Contents

**1 Part I: Understanding and proofs** **2**

1.1 I.0 Eight understanding questions . . . 2
1.2 I.1 Which losses produce probabilities? . . . 3
1.3 I.2 How much validation data does model selection need? . . . 4
1.4 I.3 Three ways to get the same linear classifier . . . 5
1.5 I.4 No classifier is best at every prevalence . . . 6

**2 Part II: Situations and experiments** **8**

2.1 II.1 Reading learning curves, with a labelling budget . . . 8
2.2 II.2 An ablation study finds a shortcut . . . 9
2.3 II.3 Labels that come only from alerts . . . 10
2.4 II.4 When the cost depends on the input: card fraud . . . 12
2.5 II.5 Aggregate metrics hide where the model fails . . . 13
2.6 II.6 Monitoring a model after deployment . . . 14

# 1 Part I: Understanding and proofs

## 1.1 I.0 Eight understanding questions

**I.0 ($\star$) 20 minutes**

Answer each in two or three sentences. Precision matters more than length.

**(1)** The posterior $p(y \mid x)$ is a property of the *problem*; the score $s_\theta(x)$ is a property of the *model*. Explain the distinction and say what connects them.

$p(y \mid x)$ is determined by the data-generating distribution alone: fix the population and the representation, and the posterior is fixed whether or not anyone builds a model. $s_\theta(x)$ is a function chosen by a learning algorithm from a family; different training sets or families give different scores for the same $x$. They are connected by estimation: a calibrated model has $\sigma(s_\theta(x)) \approx p(1 \mid x)$, and the quality of that approximation is what validation measures [one-read §2.2, 4.1].

**(2)** Why can the Bayes error be strictly positive even with infinite training data and unlimited model capacity?

The Bayes error $\mathbb{E}[\min\{p(X), 1-p(X)\}]$ depends only on how well the *representation* determines the label. If flows with the same $x$ carry different labels, $0 < p(x) < 1$ there, and any rule based on $x$ alone is wrong on some of them. More data and capacity let a learner approach the Bayes rule; they cannot add information that $x$ does not contain. Only a richer representation can lower the floor [one-read §2.2].

**(3)** The validation error of the *selected* model is a biased estimate of its population error, but its test error (used once) is not. Explain both halves.

Selection picks the configuration whose validation error is smallest, so it favours configurations that were lucky on that particular validation set; the minimum of noisy estimates is biased downward. The test set played no part in the choice, so from its point of view the selected model is just one fixed model, and the average of i.i.d. losses of a fixed model is unbiased. The guarantee evaporates the moment the test set influences any decision [one-read §3.5].

**(4)** In what sense does the capacity of $k$-NN *decrease* as $k$ grows, although the algorithm has no parameters?

Capacity is the range of functions a method can produce, not a parameter count. With $k = 1$ the prediction can change every time the nearest training point changes, so the boundary can be arbitrarily jagged; with $k = n$ every query gets the majority label, a single constant function. Larger $k$ averages more neighbours, which smooths the prediction and restricts the set of achievable boundaries: lower variance, higher bias [one-read §2.4, 3.3].

**(5)** What does "generative models make their assumptions visible" mean concretely? Give one assumption and how it would be checked.

A generative model writes down $p(x \mid y)$ explicitly, so every simplification is a statement about the data that can be tested against it. LDA assumes each class is Gaussian with a shared covariance: one can compare per-class covariance estimates, or plot a feature's histogram per class against the fitted normal. Naive Bayes assumes conditional independence: one can compute within-class correlations between features. A discriminative model makes assumptions too (a linear boundary, say), but about $p(y \mid x)$, which is harder to inspect directly [one-read §4.3, 4.6].

**(6)** Under prior shift, why does the posterior change while the class-conditional likelihoods do not?

Prior shift means $p(x \mid y)$ is the same in training and deployment and only $P(y)$ changes. The posterior is $p(y \mid x) \propto p(x \mid y)\, P(y)$, so it inherits the change through the prior factor: the same evidence $p(x \mid y)$ is weighed against a different base rate. In odds form the log-odds shift by the constant $\ln \frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)}$ [one-read §4.7].

**(7)** Why must preprocessing steps such as scaling and feature selection be fitted *inside* each cross-validation fold rather than once before cross-validation?

Cross-validation estimates the error of the whole procedure that will be applied to new data, and that procedure includes the preprocessing. If the scaler or selector is fitted on all the data first, each fold's validation rows have already influenced the model through those statistics, so the validation error measures a procedure that saw its own test data. For feature selection using labels the bias can be large [one-read §3.6].

**(8)** A per-event classifier cannot detect a port scan, but a classifier on window features can. In what sense does the second one "see time"?

Its input vector is computed from the recent history of the same source (counts, distinct ports, failure fraction, score sums over a window), so the representation itself encodes the pattern across events. The classifier is still a function of one vector, but that vector summarises a sequence; time enters through the representation, not through the learning algorithm [one-read §4.8].

## 1.2 I.1 Which losses produce probabilities?

**I.1 (⋆⋆) 15 minutes**

Fix an input $x$ with true posterior $p = P(Y = 1 \mid x)$. A model outputs a number $r \in [0, 1]$ for this $x$. Consider three per-example losses and their *expected* value over $Y$:

$$\begin{aligned} \text{cross-entropy:} \quad & \ell_{\mathrm{CE}}(y, r) = -y \ln r - (1 - y) \ln(1 - r), \\ \text{squared:} \quad & \ell_{\mathrm{sq}}(y, r) = (y - r)^2, \\ \text{absolute:} \quad & \ell_{\mathrm{abs}}(y, r) = |y - r|. \end{aligned}$$

**Tools you need**

$\mathbb{E}_Y[\ell(Y, r)] = p\,\ell(1, r) + (1-p)\,\ell(0, r)$; entropy $H(p) = -p \ln p - (1-p)\ln(1-p)$ and KL divergence $\mathrm{KL}(p \,\|\, r) = p \ln \frac{p}{r} + (1-p) \ln \frac{1-p}{1-r}$ [one-read §2.6].

**(a)** Show that the expected cross-entropy is minimised at $r = p$, and that $\mathbb{E}_Y[\ell_{\mathrm{CE}}(Y, r)] = H(p) + \mathrm{KL}(p \,\|\, r)$.

$\mathbb{E}_Y[\ell_{\mathrm{CE}}] = -p \ln r - (1-p)\ln(1-r) =: g(r)$. Then $g'(r) = -\frac{p}{r} + \frac{1-p}{1-r}$, which is 0 iff $p(1-r) = (1-p)r$ iff $r = p$; and $g''(r) = \frac{p}{r^2} + \frac{1-p}{(1-r)^2} > 0$, so this is the minimum.

Add and subtract $p \ln p + (1-p)\ln(1-p)$:

$$g(r) = \underbrace{-p \ln p - (1-p)\ln(1-p)}_{H(p)} + \underbrace{p \ln \frac{p}{r} + (1-p) \ln \frac{1-p}{1-r}}_{\mathrm{KL}(p \| r)}.$$

Since $\mathrm{KL} \geq 0$ with equality iff $r = p$, the minimum value is $H(p)$, the irreducible uncertainty of the label, and the excess loss of any model is exactly its KL divergence from the truth. ■

**(b)** Show that expected squared loss is also minimised at $r = p$.

$\mathbb{E}_Y[(Y-r)^2] = p(1-r)^2 + (1-p)r^2$. Derivative: $-2p(1-r) + 2(1-p)r = 2(r-p)$, zero at $r = p$, and the function is a convex parabola in $r$. Minimum value $p(1-p)$, the variance of a Bernoulli($p$). ■

**(c)** Show that expected absolute loss is minimised at $r \in \{0, 1\}$ (never at $p$ unless $p \in \{0, \frac{1}{2}, 1\}$). What does this mean for a model trained with absolute loss?

$\mathbb{E}_Y|Y-r| = p(1-r) + (1-p)r = p + r(1-2p)$, which is *linear* in $r$. If $p < \frac{1}{2}$ the slope is positive and the minimum is at $r = 0$; if $p > \frac{1}{2}$ the slope is negative and the minimum is at $r = 1$; at $p = \frac{1}{2}$ every $r$ ties.

A model trained to minimise absolute loss is therefore rewarded for outputting hard 0/1 answers, not probabilities: its outputs cannot be read as posteriors, and cost-sensitive thresholds, prior-shift corrections and review bands all become meaningless. Losses whose expected value is minimised by the true probability are called *proper*; cross-entropy and squared loss are proper, absolute loss is not. ■

**(d)** A colleague proposes training with the 0–1 loss on the thresholded prediction $\mathbf{1}(r \geq 0.5)$. Explain in one paragraph why this is both uninformative about probabilities and hard to optimise.

$\mathbb{E}_Y[\mathbf{1}(Y \neq \mathbf{1}(r \geq 0.5))]$ equals $1-p$ if $r \geq 0.5$ and $p$ otherwise: it is a step function of $r$, constant on each side of 0.5. Every $r$ on the correct side is equally good, so the loss carries no information about *how* probable the class is, and no calibration can be learned. The same step shape makes the empirical loss piecewise constant in the parameters: its gradient is zero almost everywhere, so gradient methods have nothing to follow. Cross-entropy is used precisely because it is a smooth, proper surrogate whose minimiser also minimises the 0–1 risk once thresholded at the Bayes threshold.

## 1.3 I.2 How much validation data does model selection need?

**I.2 (⋆⋆⋆) 20 minutes**

A fixed classifier $f$ has population error $R(f)$. On $n$ i.i.d. validation examples its empirical error is $\widehat{R}_n(f)$. Hoeffding's inequality for averages of $n$ independent $\{0, 1\}$ variables states

$$P(|\widehat{R}_n(f) - R(f)| > \varepsilon) \leq 2e^{-2n\varepsilon^2}.$$

**Tools you need**

Union bound $P(A_1 \cup \cdots \cup A_M) \leq \sum_m P(A_m)$; generalisation gap and selection [one-read §3.1, 3.5]. Values: $\ln 40 = 3.689$, $\ln 4000 = 8.294$, $\ln 400{,}000 = 12.90$.

**(a)** For a single fixed classifier, how many validation examples guarantee $|\widehat{R}_n - R| \leq 0.02$ with probability at least 0.95?

Require $2e^{-2n\varepsilon^2} \leq \delta$, i.e. $n \geq \frac{\ln(2/\delta)}{2\varepsilon^2}$. With $\varepsilon = 0.02$, $\delta = 0.05$: $n \geq \frac{\ln 40}{2\times 0.0004} = \frac{3.689}{0.0008} \approx 4{,}611$.

**(b)** Now $M$ candidate configurations $f_1, \ldots, f_M$ are evaluated on the *same* validation set and the best is selected. Show that with probability at least $1-\delta$, *every* candidate satisfies $|\widehat{R}_n(f_m) - R(f_m)| \leq \varepsilon$ provided $n \geq \frac{\ln(2M/\delta)}{2\varepsilon^2}$. Conclude that the selected $\hat{f}$ satisfies $R(\hat{f}) \leq \min_m R(f_m) + 2\varepsilon$.

Let $A_m$ be the bad event $|\widehat{R}_n(f_m) - R(f_m)| > \varepsilon$. Each candidate is a fixed function (chosen before seeing the validation data), so Hoeffding applies to each: $P(A_m) \leq 2e^{-2n\varepsilon^2}$. By the union bound, $P(\text{some } A_m) \leq 2Me^{-2n\varepsilon^2}$, which is $\leq \delta$ iff $n \geq \frac{\ln(2M/\delta)}{2\varepsilon^2}$.

On the complementary good event, every empirical error is within $\varepsilon$ of its population error. Let $f^\star$

be the candidate with the smallest population error. Then

$$R(\hat{f}) \le \widehat{R}_n(\hat{f}) + \varepsilon \le \widehat{R}_n(f^\star) + \varepsilon \le R(f^\star) + 2\varepsilon,$$

using in the middle that $\hat{f}$ minimises the empirical error. ■

The selected model is within $2\varepsilon$ of the best candidate, with one $\varepsilon$ paid for the optimism of the winner and one for not knowing which candidate is truly best.

**(c)** Compute $n$ for $M = 100$ and $M = 10{,}000$ with $\varepsilon = 0.02$, $\delta = 0.05$. Compare with (a) and comment on the growth.

$M = 100$: $n \ge \frac{\ln 4000}{0.0008} = \frac{8.294}{0.0008} \approx 10{,}368$. $M = 10{,}000$: $n \ge \frac{12.90}{0.0008} \approx 16{,}124$.

Comparing a hundred configurations costs about 2.2 times the data of comparing one, and ten thousand about 3.5 times: the requirement grows only like $\ln M$. This is the positive side of selection: trying many configurations is cheap in validation data, *provided* the candidates were fixed before looking at the validation set. The negative side appears in (d).

**(d)** A team has $n = 2{,}000$ validation flows and compares $M = 100$ configurations. What $\varepsilon$ does the bound guarantee at $\delta = 0.05$? Then they iterate: they look at the validation results, invent 100 new configurations inspired by what worked, and evaluate those too. Does the bound still apply with $M = 200$?

$\varepsilon = \sqrt{\frac{\ln(2M/\delta)}{2n}} = \sqrt{\frac{8.294}{4000}} \approx 0.046$: the winner's validation error is guaranteed only to within about 4.6 points, which is more than the differences the team is probably trying to detect.

The second round breaks the proof's assumption: the new candidates were chosen *after* seeing the validation data, so they are not fixed functions and Hoeffding does not apply to them as stated. Adaptive reuse of a validation set can produce much larger optimism than $\ln M$ suggests, because the candidates are built to exploit that set's particular noise. Remedies: a fresh validation split for the second round, cross-validation, or at least treating the second-round scores as exploratory and confirming on untouched data.

## 1.4 I.3 Three ways to get the same linear classifier

**I.3 (⋆⋆⋆) 20 minutes**

Logistic regression, LDA and Bernoulli Naive Bayes look like three different models. Show that all three produce a posterior of the form $P(y = 1 \mid x) = \sigma(w^\top x + b)$, and identify $w$ and $b$ for the two generative models.

**Tools you need**

Posterior log-odds $\ln \frac{P(1|x)}{P(0|x)} = \ln \frac{p(x|1)}{p(x|0)} + \ln \frac{\pi_1}{\pi_0}$; $\sigma(z) = 1/(1 + e^{-z})$ and $z = \ln \frac{\sigma(z)}{1-\sigma(z)}$ [one-read §4.2–4.5].

**(a)** Show that for any binary classifier, $P(y = 1 \mid x) = \sigma(z(x))$ where $z(x)$ is the posterior log-odds. Hence a model is "logistic-shaped" exactly when its log-odds is affine in $x$.

Write $q = P(y = 1 \mid x)$ and $z = \ln \frac{q}{1-q}$. Then $e^z = \frac{q}{1-q}$, so $q = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z)$. This is an identity, true for every model; what distinguishes models is the *form* of $z(x)$. Logistic regression *assumes* $z(x) = w^\top x + b$ and fits $w, b$ directly.

**(b) LDA.** With $p(x \mid c) = \mathcal{N}(\mu_c, \Sigma)$ in $d$ dimensions, shared $\Sigma$, show that $z(x) = w^\top x + b$ with $w = \Sigma^{-1}(\mu_1 - \mu_0)$ and $b = -\frac{1}{2}(\mu_1^\top \Sigma^{-1} \mu_1 - \mu_0^\top \Sigma^{-1} \mu_0) + \ln \frac{\pi_1}{\pi_0}$.

$\ln p(x \mid c) = -\frac{1}{2}(x-\mu_c)^\top \Sigma^{-1}(x-\mu_c) - \frac{d}{2}\ln 2\pi - \frac{1}{2}\ln\det\Sigma$. Expanding the quadratic, $(x-\mu_c)^\top\Sigma^{-1}(x-\mu_c) = x^\top\Sigma^{-1}x - 2\mu_c^\top\Sigma^{-1}x + \mu_c^\top\Sigma^{-1}\mu_c$. In the difference $\ln p(x \mid 1) - \ln p(x \mid 0)$ the terms $x^\top\Sigma^{-1}x$, $\frac{d}{2}\ln 2\pi$ and $\frac{1}{2}\ln\det\Sigma$ are identical for both classes and cancel, leaving

$$z(x) = (\mu_1 - \mu_0)^\top\Sigma^{-1}x - \tfrac{1}{2}(\mu_1^\top\Sigma^{-1}\mu_1 - \mu_0^\top\Sigma^{-1}\mu_0) + \ln\frac{\pi_1}{\pi_0},$$

which is $w^\top x + b$ with the stated $w$ and $b$ (using $\Sigma^{-1}$ symmetric). ■

The cancellation of $x^\top\Sigma^{-1}x$ is exactly where the shared covariance is used; with $\Sigma_0 \neq \Sigma_1$ it survives and the boundary is quadratic.

**(c) Bernoulli Naive Bayes.** With binary features and $p(x_j = 1 \mid c) = \theta_{jc}$, show that $z(x) = \sum_j w_j x_j + b$ with

$$w_j = \ln\frac{\theta_{j1}(1-\theta_{j0})}{\theta_{j0}(1-\theta_{j1})}, \qquad b = \ln\frac{\pi_1}{\pi_0} + \sum_j \ln\frac{1-\theta_{j1}}{1-\theta_{j0}}.$$

Under conditional independence, $p(x \mid c) = \prod_j \theta_{jc}^{x_j}(1-\theta_{jc})^{1-x_j}$, so

$$\ln\frac{p(x \mid 1)}{p(x \mid 0)} = \sum_j \Big[x_j \ln\frac{\theta_{j1}}{\theta_{j0}} + (1-x_j)\ln\frac{1-\theta_{j1}}{1-\theta_{j0}}\Big] = \sum_j x_j\Big[\ln\frac{\theta_{j1}}{\theta_{j0}} - \ln\frac{1-\theta_{j1}}{1-\theta_{j0}}\Big] + \sum_j \ln\frac{1-\theta_{j1}}{1-\theta_{j0}}.$$

The bracket is $w_j$; adding $\ln(\pi_1/\pi_0)$ gives $b$. ■

$w_j$ is the log-odds ratio of feature $j$: how much seeing $x_j = 1$ rather than $x_j = 0$ shifts the evidence. Naive Bayes is a linear classifier whose weights are set by counting rather than by optimisation.

**(d)** Since all three are linear, what differs between them? For each pair, say which you expect to win when (i) data are scarce and the generative assumption is roughly right, (ii) data are plentiful and the assumption is wrong, (iii) two features are near-duplicates.

They share the hypothesis class but not the *fitting criterion*: logistic regression chooses $(w, b)$ to minimise cross-entropy on $p(y \mid x)$ directly; LDA and NB set $(w, b)$ through estimates of $p(x \mid y)$ and $\pi$, which also explain how $x$ is distributed, a stronger demand.

(i) Generative wins: it uses the assumption as extra information, so its estimates have lower variance; NB in particular needs only per-feature counts.

(ii) Logistic wins: it converges to the best *linear* rule for $p(y \mid x)$ whatever $p(x \mid y)$ looks like, whereas LDA/NB converge to the rule implied by a wrong model of $x$, so their bias does not vanish with $n$.

(iii) Logistic wins: with duplicated features, NB adds the same $w_j$ twice (double counting the evidence, overconfident posteriors), LDA handles it through the covariance but needs that covariance to be well estimated; logistic regression spreads the weight between the two copies since only their sum affects the loss.

## 1.5 I.4 No classifier is best at every prevalence

**I.4 (⋆⋆)** **15 minutes**

A classifier with recall $\rho$ and false-positive rate $\phi$ is used on a population with attack prevalence $\pi$, costs $C_{FN}$ per miss and $C_{FP}$ per false alert. Two classifiers are available:

$$A:\ \rho = 0.90,\ \phi = 0.06; \qquad B:\ \rho = 0.70,\ \phi = 0.02; \qquad C_{FN} = 10,\ C_{FP} = 1.$$

**Tools you need**

Expected cost per flow as a function of the prevalence; recall and FPR do not depend on $\pi$ [one-read §3.6].

**(a)** Show that the expected cost per flow is $C(\pi) = \pi(1-\rho)C_{FN} + (1-\pi)\phi\, C_{FP}$, a straight line in $\pi$. Write it for $A$, $B$, and for the two trivial rules "never alert" and "always alert".

A random flow is an attack with probability $\pi$, in which case it is missed with probability $1-\rho$ at cost $C_{FN}$; it is benign with probability $1-\pi$, in which case it is falsely alerted with probability $\phi$ at cost $C_{FP}$. Adding the two gives $C(\pi)$, affine in $\pi$.

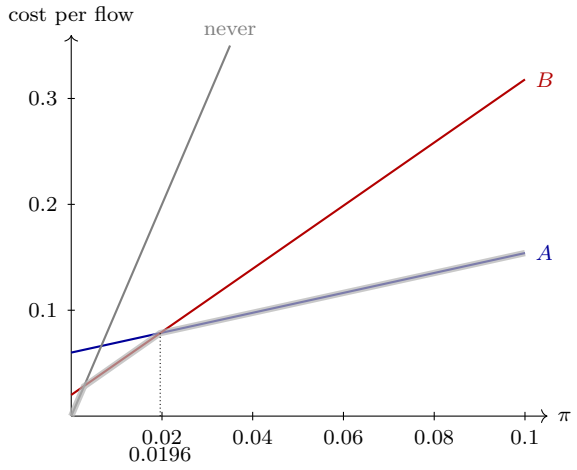
$$C_A(\pi) = 10\pi(0.1) + 0.06(1-\pi) = 0.06 + 0.94\pi,$$
$$C_B(\pi) = 10\pi(0.3) + 0.02(1-\pi) = 0.02 + 2.98\pi,$$
$$C_{\text{never}}(\pi) = 10\pi \quad (\rho = 0, \phi = 0), \qquad C_{\text{always}}(\pi) = 1-\pi \quad (\rho = 1, \phi = 1).$$

**(b)** Find the prevalence at which $A$ and $B$ have equal cost, and say which is better on each side. Then find where $A$ and $B$ each stop beating "never alert".

$0.06 + 0.94\pi = 0.02 + 2.98\pi \iff 0.04 = 2.04\pi \iff \pi \approx 0.0196$. Above 2% prevalence $A$ is cheaper (its higher recall pays off when attacks are common enough); below 2% $B$ is cheaper (its lower false-alert rate matters when almost everything is benign).

Against never alerting: $A$ wins iff $0.06 + 0.94\pi < 10\pi \iff \pi > 0.06/9.06 \approx 0.0066$; $B$ wins iff $\pi > 0.02/7.02 \approx 0.0028$. Below those prevalences, doing nothing is cheapest: the false alerts cost more than the few attacks they would catch.

**(c)** Sketch the four lines on $\pi \in [0, 0.1]$ and read off the lower envelope. What does the envelope represent?




"Always alert" is off the chart ($\approx 0.9$–1). The grey band is the lower envelope.

The envelope is the cost of the best available rule at each prevalence: never alert up to 0.28%, then $B$ up to 1.96%, then $A$. It shows that "which classifier is better" has no answer without the prevalence and the costs; a model chosen on a 9%-prevalence validation set can be the wrong one at 1% deployment, even though its recall and FPR are unchanged.

**(d)** $A$ and $B$ are in fact the *same* scoring model at two thresholds. Explain why that makes the prevalence-dependent choice easy to implement, and what should be re-estimated when the prevalence changes.

Each threshold on the same score gives one $(\rho, \phi)$ pair, i.e. one point on the ROC curve, and so one cost line. Choosing between $A$ and $B$ is just moving the threshold, which needs no retraining. When the prevalence changes, the recall and FPR of each threshold stay the same (they are within-class quantities), so the ROC curve is still valid; only the weights $\pi$ and $1-\pi$ in the cost change. Re-estimate $\pi$ on deployment data (an audit sample), recompute the expected cost of each threshold,

and move to the cheapest. If the scores are calibrated, the same thing is achieved by correcting the posterior for prior shift and keeping the Bayes threshold $C_{FP}/(C_{FP}+C_{FN})$.

# 2 Part II: Situations and experiments

## 2.1 II.1 Reading learning curves, with a labelling budget

**II.1 (⋆⋆)** **20 minutes**

Three models were trained on nested subsets of a flow dataset. Each cell is *training error / validation error*, validation on a fixed held-out set of 5,000 flows.

| $n$ | 250 | 1,000 | 4,000 | 16,000 |
|---|---|---|---|---|
| logistic regression | 0.18 / 0.21 | 0.19 / 0.20 | 0.19 / 0.195 | 0.19 / 0.192 |
| 5-NN | 0.05 / 0.30 | 0.06 / 0.22 | 0.07 / 0.17 | 0.07 / 0.14 |
| flexible model | 0.00 / 0.33 | 0.01 / 0.24 | 0.02 / 0.16 | 0.03 / 0.12 |

Labelling costs money: the team can afford 16,000 more labelled flows (to reach 32,000) or none.

**Tools you need**

Generalisation gap, bias and variance as functions of sample size, underfitting and overfitting [one-read §3.1–3.3].

**(a)** For each model, describe the behaviour of the training error, the validation error and the gap as $n$ grows. Diagnose each as bias-dominated or variance-dominated at $n = 16{,}000$.

- **Logistic regression.** Training and validation errors converge quickly to about 0.19; the gap is nearly closed. More data no longer helps: the model is *bias-dominated*. A linear boundary cannot represent the problem better than 0.19.
- **5-NN.** Training error stays low (a point is usually its own neighbour's class); validation error falls steadily, $0.30 \to 0.14$, and the gap is still 0.07. *Variance-dominated*, still improving.
- **Flexible model.** Training error near zero, validation falling fastest, $0.33 \to 0.12$, gap 0.09 and shrinking. *Variance-dominated*, with the lowest bias of the three: at every $n \geq 4{,}000$ it beats the others despite the largest gap.

**(b)** Extrapolate each model's validation error to $n = 32{,}000$ and justify the extrapolation. Which model should get the labelling budget?

Read the trend per doubling-of-doubling: each column multiplies $n$ by 4.

- Logistic: $0.21, 0.20, 0.195, 0.192$: the decrements are shrinking towards a floor near 0.19. At 32,000, about 0.19. Spending the budget here buys nothing.
- 5-NN: decrements $0.08, 0.05, 0.03$ per quadrupling, so roughly 0.015–0.02 per doubling now; expect about 0.12–0.125 at 32,000.
- Flexible: decrements $0.09, 0.08, 0.04$; expect about 0.10 at 32,000 (a doubling is half a column step, and the last column step was 0.04).

Give the budget to the flexible model: it has the lowest current error, the largest remaining gap, and the steepest curve. The extrapolations are rough (curves flatten unpredictably), so the honest statement is "the flexible model is expected to reach roughly 0.10, the others cannot".

**(c)** The training error of the flexible model *rises* with $n$ ($0.00 \to 0.03$). A colleague says this shows the model is getting worse. Respond.

The opposite. With 250 points the model can memorise every one, including label noise and coinci-

dences; with 16,000 it can no longer fit all of them and is forced to fit the regularities instead. Rising training error together with falling validation error is the signature of a high-capacity model being tamed by data: the gap closes from both sides. What would be worrying is training error rising while validation error stalls, which would suggest the model's capacity is now the limit.

**(d)** Rather than labelling 16,000 random flows, the team could label 16,000 flows chosen because the current flexible model is *uncertain* about them (posterior near 0.5). Give one reason this could help and one way it could hurt the validation measurement.

Help: uncertain flows lie near the current boundary, where more examples refine the model most; random flows mostly land in regions already well classified. Hurt: the new training data are no longer a sample from the deployment distribution, so training-set statistics (prevalence, feature distributions, the intercept of a logistic model) become biased, and if any of these actively selected flows leak into validation, the validation set stops representing deployment too. The validation set must remain a random sample, and prevalence-dependent quantities must be re-estimated on it, not on the training set.

## 2.2 II.2 An ablation study finds a shortcut

**II.2 (⋆⋆) 20 minutes**

A flow classifier uses six features. The team runs an *ablation*: retrain with one feature removed at a time, same split, same hyperparameters. Validation error:

| removed feature | none | bytes | duration | dst port | TCP flags | hour of day | source subnet |
|---|---|---|---|---|---|---|---|
| validation error | 0.040 | 0.050 | 0.045 | 0.060 | 0.050 | **0.210** | **0.090** |

Later, the full model is evaluated on flows captured during a different week, in which the red team ran its attacks during office hours from several subnets: error 0.35. The model without *hour of day* scores 0.08 on that same week.

**Tools you need**

Shortcuts versus leakage, data-collection artefacts, experiments that distinguish them [one-read §1.5, 3.6].

**(a)** Interpret the ablation table on its own. Which features does the model rely on most? Is anything suspicious from the table alone?

Removing *hour of day* raises the error five-fold, removing *source subnet* doubles it; the other four each cost half a point to two points. So the model leans heavily on when the flow happened and where it came from. From the table alone this is not proof of a problem: a feature can be legitimately strong. It is suspicious because neither feature describes what the flow *does*; both describe the circumstances of its capture, and a classifier that mostly needs the clock and the subnet is a classifier that may have learned how the dataset was collected.

**(b)** Explain what the second week shows, and why the ablation on the original split could not have revealed it by itself.

On the new week the full model fails (0.35) while the model without the hour feature works (0.08). So the hour feature was a *shortcut*: in the original data, attacks were run at a particular time (a red-team campaign overnight, say), the model learned "night ⇒ attack", and that rule has no validity when attacks happen in office hours. The subnet feature is probably a second, weaker shortcut of the same kind (attacks launched from a few machines).

The original validation set was drawn from the same collection, so the shortcut held there too: the

hour feature was *genuinely predictive on that distribution*. Ablation measures how much a feature helps on the available distribution; it cannot tell a real signal from an artefact that is present on both sides of the split. Only data collected under different circumstances can.

> **(c)** Is the hour feature target leakage? Argue carefully.

No. Leakage means using information that will not be available at decision time, or that is a consequence of the label. The hour of a flow is available at decision time and is not caused by the label. The problem is different: the *association* between hour and label in the training data is an artefact of how the attack data were generated, so it does not transfer. A leak makes the model unusable in deployment because the feature is missing; a shortcut makes it unreliable because the feature means something else there. The fix for leakage is always to drop the feature; the fix for a shortcut may be to drop it, or to collect data in which the association is broken.

> **(d)** Design two experiments, using only the *original* data, that would have raised the alarm before the second week.

1. **Per-class feature distributions.** Plot the histogram of hour of day for attacks and for benign flows. A clean separation (attacks all between 01:00 and 04:00) is implausible for real attacks and points directly at the collection process. Do the same for source subnet.
2. **Split by the suspected artefact.** Hold out a validation set consisting of a different time-of-day band, or different subnets, from the training set (grouped split by hour band or by subnet). If the full model's error jumps on that split while the hour-free model's does not, the dependence on circumstance is exposed without new data.

A third option is a *single-feature* model: a classifier using the hour alone reaching, say, 0.06 error would show that the clock nearly solves the dataset, which no genuine intrusion problem allows.

> **(e)** The team decides to keep the hour feature "because attacks really are more common at night in production". Under what conditions is that defensible, and what must they measure?

Defensible only if the association exists in the *deployment* distribution and is stable: a production base rate that genuinely varies with time of day is legitimate information, exactly like a prior that changes by hour. They must measure it there, not in red-team data: estimate the attack rate by hour band from audited production traffic, check that the model's reliance matches that rate rather than the red team's schedule, and monitor it, because attackers adapt to it the moment it is known (Problem 7 of the extra-practice set). Even then the hour should act as a prior-like adjustment, not as the dominant feature it was in the ablation.

## 2.3 II.3 Labels that come only from alerts

**II.3 (⋆⋆⋆) 25 minutes**

A deployed detector processes 100,000 flows per day and alerts on about 2,000. Analysts investigate every alert and record a verdict: on a typical day 300 alerts are confirmed attacks. The verdicts are stored and used as labels to retrain the model every month. Nobody looks at the 98,000 un-alerted flows.

**Tools you need**

Precision and recall from counts; what a label process observes and what it misses [one-read §1.3, 3.6]; Poisson rule of thumb: a count $k$ has standard deviation about $\sqrt{k}$.

> **(a)** Which metrics can be computed from the analysts' verdicts, and which cannot? Compute what can be.

Among the alerted flows the verdicts give $TP = 300$ and $FP = 1{,}700$, so **precision** $= 300/2000 =$

0.15 is measurable. **Recall** is not: it needs $FN$, the attacks among the 98,000 un-alerted flows, which nobody labels. Neither is the **prevalence**, nor accuracy, nor the false-positive rate (which needs the total number of benign flows). The label process only observes the region the current model already flags.

> **(b)** Describe what goes wrong when the monthly retraining uses only these verdicts as training data.

- **Selection by the old model.** Every labelled flow has posterior above the old threshold. The new model sees no examples below it, so it learns the old model's boundary rather than the problem's: attacks that the old model misses are never labelled, never trained on, and keep being missed. The errors are self-perpetuating.
- **Wrong prevalence.** The training set has 15% attacks (300 of 2000), far above the true rate, so the new model's intercept and posteriors are biased towards attack, unless corrected.
- **Analyst noise becomes the target.** Verdicts contain analyst mistakes and "could not determine" cases; the model learns the analysts, including their blind spots.
- **Feedback loop.** If alerts also change behaviour (blocked sources go quiet), the data distribution shifts in response to the model.

> **(c)** A random audit labels 1,000 of the un-alerted flows and finds 4 attacks. Estimate the daily number of missed attacks, the recall, and the prevalence. Give a rough uncertainty on each.

The audit suggests a miss rate of $4/1000 = 0.4\%$ among un-alerted flows, so $FN \approx 0.004 \times 98{,}000 \approx 392$ per day. Then

$$\text{recall} \approx \frac{300}{300+392} \approx 0.43, \qquad \text{prevalence} \approx \frac{300+392}{100{,}000} \approx 0.7\%.$$

Uncertainty: a count of 4 has standard deviation about 2, so the plausible range is roughly 0–8 attacks in the audit, i.e. $FN$ from about 0 to 800 and recall from about 0.27 to 1. The estimate says the detector probably misses more than half of attacks, but 4 events cannot pin it down.

> **(d)** How large should the audit be to estimate the number of missed attacks to within about $\pm 25\%$? Propose a sampling design that reaches that precision more cheaply than uniform random sampling.

Relative standard deviation of a count is $1/\sqrt{k}$, so $\pm 25\%$ needs $k \approx 16$ attacks found. At 0.4% that is $16/0.004 = 4{,}000$ un-alerted flows audited per day, four times the pilot.

Cheaper: **stratified sampling by score**. The un-alerted flows have posteriors from 0 to the threshold; the missed attacks are concentrated just below it. Audit heavily in the band just below the threshold (where attacks are dense) and lightly in the bulk near zero, then weight each stratum's finding by its size. Fewer audited flows yield the same number of discovered attacks and a tighter estimate; the weighting keeps the estimate unbiased. This is the same idea as importance sampling.

> **(e)** Design a labelling and retraining protocol that fixes (b). State what is labelled, how the training set is weighted, and how the next model is evaluated.

1. **Label two streams:** all alerts (as now) *and* a random (or stratified) audit sample of un-alerted flows, recording for each flow the probability with which it was selected for labelling.
2. **Weight by inverse selection probability** when training and when computing metrics: an audited flow from the bulk, selected with probability 1/100, stands for 100 flows. This removes the selection bias of the old model and restores the true prevalence.
3. **Evaluate on the audit stream only**, with the same weights, since it is the only part sampled from the deployment distribution; alerts-only metrics are reported separately as precision.
4. **Time-order everything:** train on past months, validate on the most recent one, so that drift and the feedback loop are measured rather than hidden.

5. **Keep some decisions randomised:** alert on a small random fraction of low-score flows (or route them to a cheap check), so that labels keep arriving from below the threshold and the model never becomes blind to its own misses.

## 2.4 II.4 When the cost depends on the input: card fraud

**II.4 (⋆⋆)** **15 minutes**

A card-fraud model outputs a calibrated posterior $p$ that a transaction is fraudulent. Blocking a legitimate transaction costs a fixed $C_{FP} = 5$ (customer friction). Letting a fraudulent transaction through costs its amount $a$, which is a *feature* of the transaction. Four transactions:

| | $T_1$ | $T_2$ | $T_3$ | $T_4$ |
|---|---|---|---|---|
| amount $a$ | 20 | 500 | 5 | 80 |
| posterior $p$ | 0.25 | 0.02 | 0.40 | 0.05 |

**Tools you need**

Bayes decision with input-dependent costs: compare $R(\text{block} \mid x) = C_{FP}(1-p)$ with $R(\text{allow} \mid x) = a\,p$ [one-read §2.2].

**(a)** Derive the threshold as a function of the amount, $t(a)$, and decide the four transactions.

Block iff $5(1-p) < ap \iff 5 < p(a+5) \iff p > t(a) = \dfrac{5}{5+a}$.

| | $a$ | $p$ | $t(a)$ | $R(\text{block})$ / $R(\text{allow})$ | decision |
|---|---|---|---|---|---|
| $T_1$ | 20 | 0.25 | 0.200 | 3.75 / 5.00 | block |
| $T_2$ | 500 | 0.02 | 0.0099 | 4.90 / 10.0 | block |
| $T_3$ | 5 | 0.40 | 0.500 | 3.00 / 2.00 | allow |
| $T_4$ | 80 | 0.05 | 0.059 | 4.75 / 4.00 | allow |

$T_2$ is blocked at a posterior of 2% and $T_3$ is allowed at 40%: the ranking by posterior and the ranking by action are different things once costs vary with $x$.

**(b)** A colleague wants to add the amount as a feature to the model instead, keep a single global threshold of 0.5, and let the model "learn" that large amounts matter. Explain why this does not achieve the same thing.

Adding the amount as a feature changes the model's estimate of $p(\text{fraud} \mid x)$, which is a statement about *how likely* fraud is given the amount. The decision rule needs something else: how *costly* a miss is given the amount. Even a perfect posterior at 0.02 for $T_2$ is below 0.5, so a global threshold allows it, although the expected loss of allowing is twice that of blocking. Probabilities and costs are different objects, and the model estimates only the first; the cost enters at decision time and nowhere else. (Including the amount as a feature may still be a good idea for *estimation*; it just does not replace the cost-dependent threshold.)

**(c)** Fraud losses are reimbursed by the card network above $a > 200$, so the bank's cost of a missed large fraud is capped at 200. Redo $T_2$. Then explain why the model did not need retraining.

Effective miss cost $\min(a, 200) = 200$: $t = 5/205 \approx 0.024$, and $p = 0.02 < 0.024$, so $T_2$ is now *allowed* (risks: block 4.90, allow $200 \times 0.02 = 4.00$). The posterior is unchanged; only the decision layer was edited. Separating estimation from decision means a policy change like this is a one-line change with no data, no retraining and no re-validation of the model, only of the decision rule.

**(d)** Customer friction is not really constant: for a customer who has been blocked twice this month, a third block costs 25 (they may leave). Where in the system does this belong, and what data would you need to set it?

In the cost $C_{FP}(x)$, which now depends on customer history, another feature available at decision time. The threshold becomes $t = C_{FP}(x)/(C_{FP}(x) + \min(a, 200))$, raising the bar for repeat-blocked customers. Setting the number 25 needs data on what blocks actually cost: churn rates after one, two, three blocks, and the lifetime value of a customer; that is a business measurement, not something the fraud model can learn from fraud labels. The pattern is general: the model learns probabilities from labels, the costs come from elsewhere, and the decision rule combines them per transaction.

## 2.5 II.5 Aggregate metrics hide where the model fails

**II.5 (⋆⋆)** **15 minutes**

A detector is evaluated on one day of traffic from two network segments:

| segment | flows | attacks | recall | FPR |
|---|---|---|---|---|
| office workstations | 90,000 | 100 | 0.90 | 0.01 |
| server farm | 10,000 | 400 | 0.50 | 0.01 |

The report to management says: "overall false-positive rate 1%, overall accuracy 98.8%".

**Tools you need**

Counts from rates, aggregation across groups, per-group thresholds [one-read §3.6].

**(a)** Compute $TP$, $FP$ and precision for each segment, then the overall recall, precision, FPR and accuracy. Check the report's numbers.

Office: benign 89,900, $TP = 0.9 \times 100 = 90$, $FP = 0.01 \times 89{,}900 = 899$, precision $90/989 \approx 0.09$. Server farm: benign 9,600, $TP = 0.5 \times 400 = 200$, $FP = 96$, precision $200/296 \approx 0.68$.

Overall: $TP = 290$, $FP = 995$, attacks 500, benign 99,500. Recall $290/500 = 0.58$; precision $290/1285 \approx 0.23$; FPR $995/99{,}500 = 0.01$; accuracy $(290 + 98{,}505)/100{,}000 \approx 0.988$. The report's two numbers are correct, and they are the two least informative numbers available.

**(b)** Explain, in terms of where the attacks and the benign traffic sit, why the overall recall is 0.58 although one segment has recall 0.90, and why the office precision is so low although its recall is high.

Recall is a weighted average of the per-segment recalls, weighted by *attacks*: 80% of the attacks are in the server farm, where recall is only 0.50, so $0.58 = 0.2 \times 0.9 + 0.8 \times 0.5$. The model is good exactly where attacks are rare and poor where they are common. Office precision is low because precision depends on the base rate: 100 attacks among 90,000 flows means even a 1% false-positive rate produces nine false alerts for every true one. Same FPR, same recall, and precision ranges from 0.09 to 0.68 purely because the two segments have different prevalence (Problem I.4 again).

**(c)** Analysts complain that "nine out of ten office alerts are noise" while security complains that "half the server attacks get through". Propose a change that addresses both without retraining, and say what it costs.

Use **segment-specific thresholds**. Raise the threshold on office traffic: fewer office alerts, higher office precision, at the price of some office recall (which was 0.90 on only 100 attacks, so the absolute loss is small). Lower the threshold on server-farm traffic: higher recall where the 400 attacks are, at the price of more server-farm false alerts, which are cheap in precision terms because the prevalence

there is 4%. The cost of this is that the two segments now have different operating points, which must be chosen and monitored separately, and the analysts' load shifts from office noise to server-farm alerts. Equivalently, add the segment as a feature (or as a prior) so that the model's posterior accounts for the prevalence difference and a single cost threshold does the right thing.

**(d)** Generalise: what should every evaluation report include besides the overall metrics, and why does accuracy appear in the report at all?

Per-group metrics for every group that differs in prevalence or in cost (segment, time band, attack family, customer type), each with counts, not only rates, so that the weight of each group is visible; plus the confusion matrix itself, from which any metric can be recomputed. Accuracy appears because it is the most flattering number available when one class is 99.5% of the data; it would be 99.5% for a detector that never alerts. A report that leads with it is, charitably, not written for decision-making.

## 2.6 II.6 Monitoring a model after deployment

**II.6 (⋆⋆⋆) 20 minutes**

A calibrated detector was deployed in January with a cost-based threshold of 0.10. A random audit of 2,000 flows per month provides labels. Monthly figures:

| | Jan | Feb | Mar | Apr | May | Jun |
|---|---|---|---|---|---|---|
| mean predicted posterior on audit | 0.012 | 0.012 | 0.013 | 0.013 | 0.014 | 0.014 |
| attack rate in audit | 0.012 | 0.011 | 0.014 | 0.019 | 0.025 | 0.031 |
| recall on audit | 0.82 | 0.80 | 0.74 | 0.63 | 0.55 | 0.47 |
| FPR on audit | 0.015 | 0.015 | 0.014 | 0.014 | 0.014 | 0.013 |

**Tools you need**

Calibration on average, prior shift versus concept drift, time-ordered validation [one-read §3.6, 4.7].

**(a)** Describe what is drifting and what is not. Is this prior shift, a change in $p(x \mid y)$, or both? Use the rows to argue.

The attack rate rises from 1.2% to 3.1%, so the prior is shifting. But the model's mean predicted posterior barely moves ($0.012 \to 0.014$): it has not noticed, which is expected since it was calibrated on January data. Recall falls from 0.82 to 0.47 while FPR stays flat. If *only* the prior had shifted, the model's recall and FPR, which are within-class quantities, would stay the same. Falling recall at constant FPR means the *attacks* are changing: new attacks look less like the January attacks, so they score below the threshold. Both are happening: more attacks (prior shift) *and* different attacks (a change in $p(x \mid y = 1)$, often called concept drift). The benign traffic looks stable, since FPR is constant.

**(b)** In June, what fraction of the audited attacks are missed, and about how many attacks per month are missed in production if there are 3 million flows per month? What is the model's calibration error in June?

Missed fraction $1 - 0.47 = 0.53$. June attacks in production: $0.031 \times 3{,}000{,}000 = 93{,}000$, of which about 49,000 are missed. Calibration: the model predicts an average posterior of 0.014 while the observed rate is 0.031; it under-predicts attack probability by a factor of more than two, so every cost-based decision is now made with the wrong posterior as well as the wrong boundary.

**(c)** Two fixes are proposed: (i) apply the prior-shift correction with $\pi_d = 0.031$ and keep the model; (ii) retrain on January–June data. Say what each fixes and what it cannot fix.

(i) The correction shifts every log-odds by $\ln \frac{0.031/0.969}{0.012/0.988} \approx \ln 2.63 \approx 0.97$. It restores average calibra-

tion and, with the threshold held at 0.10, lowers the effective score threshold, so recall rises and FPR rises too. It is cheap and immediate. It cannot fix the ranking: attacks that score like benign traffic stay below any sensible threshold, because the model does not recognise them. A prior correction cannot teach a model what new attacks look like.

(ii) Retraining on data that include the new attacks (from the audit labels) can change the boundary and recover recall on them. It cannot fix what has not been labelled: with 2,000 audits a month at 3% attacks, there are only about 60 labelled attacks per month, and the new families may be a handful of those. It also needs the prevalence handled correctly (the training mix is not the deployment mix) and should be validated on the most recent month, not on a random split of the six months, or the drift it is meant to address is averaged away.

**(d)** Design the monitoring protocol you would have wanted from January: what to compute monthly from the audit, what thresholds trigger action, and how to validate a retrained model before it replaces the old one.

1. **Monthly from the audit:** attack rate; mean predicted posterior versus observed rate (average calibration), ideally per score bin; recall and FPR at the deployed threshold; the score distribution of the audited attacks (are new attacks scoring low?); per-segment versions of all of these (Problem II.5).
2. **Triggers:** a prior change beyond the audit's noise (with 2,000 audits, a rate of 1.2% is $\pm 0.25$ points, so 1.9% in April is already a clear signal); a calibration ratio outside, say, $[0.7, 1.4]$; recall dropping more than a few points below January on two consecutive months. Prior shift alone triggers the cheap correction (i); falling recall triggers investigation and retraining.
3. **Validating a retrained model:** train on months 1 to $k-1$, test on month $k$ (the most recent, untouched), for several $k$, to measure how it holds up one month ahead; compare with the old model on the same months; check calibration and recall per attack family present in the audits. Deploy as a shadow model first (scoring but not acting) for a month, with its decisions audited, before it replaces the old one.
4. **Keep the audit random and sized for the decisions it must support:** 2,000 flows yield about 60 attacks a month at 3%, enough for recall to $\pm 6$ points, not enough for per-family analysis; stratify by score if finer resolution is needed (Problem II.3).